

An Application for Federated Learning of XAI Models in Edge Computing Environments

Alessio Bechini, Mattia Daole, Pietro Ducange, Francesco Marcelloni, Alessandro Renda
 Department of Information Engineering, University of Pisa, Largo Lucio Lazzarino 1, 56122 Pisa, Italy
 Email: {alessio.bechini, pietro.ducange, alessandro.renda, francesco.marcelloni}@unipi.it,
 mattia.daole@phd.unipi.it

Abstract—The next generation of wireless networks will feature an increasing number of connected devices, which will produce an unprecedented volume of data. Knowledge extraction from decentralized data imposes the exploitation of computing and learning paradigms able to tame the complexity of the network and meet the growing requirement of trustworthiness. In this regard, edge computing overcomes the limitations of cloud computing by moving virtualized computing and storage resources closer to data sources. Furthermore, Federated Learning has been recently proposed as a means to let multiple parties collaboratively train an ML model without disclosing private data. In this paper, we propose an application that enables Federated Learning of eXplainable AI models (Fed-XAI) in an edge computing environment. The proposal represents a step forward towards the adoption of trustworthy AI in next generation wireless networks, ensuring both privacy preservation and explainability. The application components are described, along with the workflow for the training and inference stages. Finally, we discuss the application deployment, in a simulated setting, for addressing a task of video streaming Quality of Experience forecasting in a vehicular network case study.

Index Terms—Federated Learning, Explainable Artificial Intelligence, Edge Computing

I. INTRODUCTION

Since the introduction of the first generation (1G) of wireless cellular technology in the 1980s, continuous efforts have been made to progressively enhance communication technologies, culminating in the deployment of the fifth-generation (5G) started in 2020 and still underway. Actually, research on beyond 5G (B5G) and 6G technologies has already kicked-off worldwide: the European Commission, for example, has funded several projects on this topic, including *HEXA-X* and *HEXA-X-II*¹.

The next generation of wireless networks will be characterized by an increased complexity due to the increased number of connected devices: the availability of massive amounts of data generated by application and network systems will

provide new advanced services and analytics opportunities [1]. Unquestionably, the integration of Artificial Intelligence (AI) and Machine Learning (ML) techniques in future networks will be crucial for the exploitation of such a huge amount of data. Moreover, it is worth noticing that big data sources will no longer be located only on megascale cloud datacenters, but will rather shift towards a distributed environment. Indeed, large volumes of data will be generated by widespread end devices, according to the mobile and Internet-of-Things (IoT) paradigms [2]. Such technological trend will also affect the way distributed computations are carried out over a networked system: the *cloud computing* paradigm, in which data are typically transmitted to a centralized data center in the network core, turns out to be unsuitable to meet stringent latency and privacy requirements. At the same time, the implementation and deployment of cumbersome AI/ML pipelines cannot burden the on-device computing capability, which is typically restricted, with limited performance and energy efficiency. Hence, *edge computing* has been recently proposed for enabling cloud services in proximity of the data sources [3].

In the context of edge computing, Multi-access Edge Computing (MEC) is a prime concept for network architectures, and developing MEC technologies is considered a key step towards 6G networks [4]. The MEC principle relies in bringing the advantages of cloud computing to the edge of the cellular network, by leveraging virtualization techniques (i.e., virtual machines and containers), and by providing computational and storage capabilities with very low latency [5]. Furthermore, the MEC host provides a set of standardized API to manage the creation of virtual resources and to interconnect the applications installed therein with the cellular domain. Thus, MEC represents a technical enabler towards distributed AI, and can be considered an innovative architectural solution for the AI-as-a-Service (AIaaS) paradigm [6], [7], originally conceived in the context of cloud computing [8]. AIaaS is a general framework to offer AI-oriented services on demand to network users. As a consequence, the deployment cost of AI services will be significantly reduced, and the use of AI-empowered applications will be increasingly widespread and accessible.

The new architectural and algorithmic solutions related to AIaaS are not only driven by increasingly stringent constraints in terms of *Key Performance Indicators* (e.g., computation and communication efficiency), but also by a number of *Key Value Indicators*, or “soft goals”, dictated by specific societal needs.

This work has been partly funded by the European Commission through the project Hexa-X (Grant Agreement no. 101015956) under the H2020 programme, by the PNRR - M4C2 - Investimento 1.3, Partenariato Esteso PE00000013 - “FAIR - Future Artificial Intelligence Research” - Spoke 1 “Human-centered AI” and the PNRR “Tuscany Health Ecosystem” (THE) (Ecosistemi dell’Innovazione) - Spoke 6 - Precision Medicine & Personalized Healthcare (CUP I53C22000780001) under the NextGeneration EU programme, and by the Italian Ministry of Education and Research (MIUR) in the framework of the FoReLab and CrossLab projects (Departments of Excellence)

¹<https://hexa-x.eu/> and <https://hexa-x-ii.eu/>, last visited May 2023

In particular, the entire chain of stakeholders, from service providers down to end-users, devotes increasing attention to the issue of *trustworthiness* of AI solutions. The concept of *trustworthy AI* has been widely considered by government agencies and regulatory bodies: for example, the European Commission in 2019 promoted the definition of the “Ethics guidelines for trustworthy AI” [9], later complemented by the “White Paper on Artificial Intelligence: a European approach to excellence and trust”² (2020) and by the “AI ACT”³ (2021). Ultimately, transparency of AI systems and user privacy emerge as crucial requirements for trustworthiness.

Privacy preservation is one of the main drivers for devising alternative solutions to data centralization. In fact, data owners are often reluctant to share their data with other parties, also within the perimeter of the legitimate interests of companies or organizations. Fortunately, ML models can be trained in a collaborative fashion even without data centralization. Indeed, Federated Learning (FL) has been recently proposed [10] for enabling the training of ML models keeping data at their own source places, but still exploiting the overall information present in decentralized datasets. The key idea of FL is to learn local AI models from local data, and then aggregate the (locally-computed) models (or their updated versions) to generate a global aggregated model. Avoiding the full access to raw data at one location by actors placed elsewhere significantly contributes to preserve the privacy of data owners.

Transparency refers to the capability of understanding the structure of a model, and is strongly related to its inherent interpretability. The ability to understand the model itself and the choices taken in the inference process, is regarded as crucial, also for legal concerns. In the context of explainable AI (XAI) [11], the aim is precisely to endow AI systems with this property, either through the design of inherently explainable models or through the adoption of post-hoc techniques.

In one of our previous papers, we have introduced the concept of Fed-XAI [12], which stands for *Federated learning of XAI models* and is conceived to provide a leap forward toward trustworthy AI. However, so far most efforts in this area have been devoted to algorithmic aspects (i.e., the adaptation of learning algorithms of interpretable models according to the federated paradigm). Conversely, little attention has been paid to technological aspects.

The objective of this work is to propose an application that supports the collaborative learning of XAI models, according to the FL paradigm, and their exploitation for inference purposes. To support XAI models, an open source FL framework is adopted and extended. In addition, for the sake of compliance with a MEC infrastructure, the actual implementations of the modules of the proposed application make use of lightweight virtualization techniques. As a paradigmatic case study, we also discuss the deployment of the application supporting Fed-XAI capability on a server for the task of forecasting Quality of Experience of video streams.

The rest of the paper is organized as follows: Section II provides some background on the Fed-XAI topic and on the interplay between FL and edge computing. Section III describes the proposed Fed-XAI application. Details about the deployment of the application for a real-world case study are reported and discussed in Section IV. Finally, Section V draws proper concluding remarks.

II. BACKGROUND AND RELATED WORKS

In this section, we first discuss the key concepts regarding Fed-XAI, with a focus on the classes of models that will constitute the ML core of our application. Then, we discuss the interplay between FL and edge computing.

A. Fed-XAI: Federated Learning of XAI models

Since the introduction of FL by Google in 2017 [13], most of the works on the subject have revolved around the well-established Federated Averaging (FedAvg) algorithm, which is immediately suitable for handling collaborative learning of models optimized through Stochastic Gradient Descent (SGD). Neural network models, for instance, belong to this category, and indeed they have been extensively investigated in the context of FL. However, they are generally considered as “opaque”, i.e. difficult to interpret. Other classes of models, and in particular those inherently interpretable such as Decision Trees and Rule-based Systems, are not typically learned through the optimization of a differentiable global objective function (e.g., through SGD), and therefore require ad-hoc training strategies for the federated setting, necessarily different from the classic FedAvg.

Takagi-Sugeno-Kang Fuzzy RBSs (TSK-FRBSs) [14] are considered among the most interpretable regression models in ML. Furthermore, they proved to be suitable to deal with noise or uncertainty thanks to the adoption of concepts from fuzzy set theory. Typically, TSK-FRBS are learned with a two-stage approach based on structure identification and model parameter identification. During structure identification, the number of rules and the conditional part of the rules are determined; this is done either with grid-partitioning of the input space, or by exploiting fuzzy clustering methods. In the model parameter identification stage, parameters of the local linear models are learned, typically by pseudo-inversion or by applying the recursive least square method. Rules of first-order TSK-FRBSs are in the following form:

$$R_k : \mathbf{IF} \ X_1 \text{ is } A_{1,j_{k,1}} \ \mathbf{AND} \ \dots \ \mathbf{AND} \ X_F \text{ is } A_{F,j_{k,F}} \\ \mathbf{THEN} \ y_k = \gamma_{k,0} + \sum_{i=1}^F \gamma_{k,i} \cdot x_i \quad (1)$$

where $\{X_1, X_2, \dots, X_F\}$ is the set of input variables and $\{A_{f,1}, A_{f,2}, \dots, A_{f,T_f}\}$ is a fuzzy partition over the universe of discourse of the variable X_f with T_f fuzzy sets ($f = 1, 2, \dots, F$), each labeled with a linguistic term; y_k is the associated output and is evaluated as the linear combination of the elements of an input pattern $\mathbf{x} = [x_1, x_2, \dots, x_F]^T$, parameterized by the vector of coefficients $\gamma_k = \{\gamma_{k,0}, \gamma_{k,1}, \dots, \gamma_{k,F}\}$.

²<https://shorturl.at/grtBU>, last visited May 2023

³<https://artificialintelligenceact.eu/the-act/>, last visited May 2023

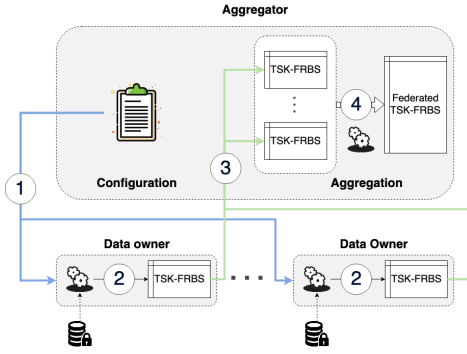


Fig. 1. Overview of the proposed approach for FL of TSK-FRBSs.

Naturally, at inference stage, an input pattern can activate more than one rule, each with a different firing strength resulting from the interaction between the crisp input and the antecedent membership functions: typically, the predicted output is evaluated as the weighted average of the outputs associated with the activated rules. As an alternative, the maximum matching paradigm considers only the rule with the highest firing strength and entails a higher level of interpretability, compared to the weighted average one, as the final output can be motivated by looking just at one single rule.

TSK FRBSs are among the most studied regression models in Fed-XAI [15], [16]. In particular, an highly interpretable learning scheme for TSK-FRBSs has been recently proposed [17]. An overview of this approach is shown in Fig. 1.

The procedure consists of the following four steps: (1) the aggregator transmits the configuration parameters (e.g., number of fuzzy sets for input space grid partitioning) to the data owners, or clients; (2) the data owners generate the local TSK-FRBS based on their own private data; (3) each data owner transmits the local TSK-FRBS to the aggregator; (4) the aggregator generates the global Federated TSK-FRBS by aggregating the local models. In a nutshell, the global rule base is created as follows: first, all the rules collected from the data owners are juxtaposed in a comprehensive rule base. Subsequently, the rule base is polished by skimming conflicting rules (i.e., rules from different sources, with the same antecedent but different consequents): the set of conflicting rules is replaced with a single rule having the same antecedent and a novel consequent obtained by averaging the coefficients of the consequents of the conflicting rules. The averaging operation is weighted: each rule, in fact, has an associated rule weight computed as the harmonic mean of its support (indicating how much a rule is activated by the samples of the training set), and confidence (indicating the average quality of the predictions of the rule measured on the training set). Finally, the global model is ready to be deployed at the data owners' side. In the inference stage, the maximum matching strategy is adopted, so to enhance the local interpretability.

B. Federated Learning in Edge Computing environment

Edge Computing (EC) is generally considered a key technology for enabling the deployment of distributed AI [18].

The EC paradigm encompasses three layers, namely a cloud computing center, edge servers, and IoT devices. Thanks to the intermediate layer, EC moves computing services from the centralized approach typical of cloud computing to the edge of the network, i.e., closer to data sources, thus reducing latency and load on the network [19].

Multi-access Edge Computing (MEC)⁴ provides standard indications for implementing an EC infrastructure, relying on *virtualized components* to ensure flexibility in deployment and exploitation. In the MEC architecture, *MEC hosts* (i.e., servers) are coupled with *base stations* (i.e., telecommunication nodes that facilitate wireless communication in 5G/6G networks) spread over a geographic area. MEC hosts offer a virtualized infrastructure at network edge, supporting the deployment of applications. Lightweight virtualization techniques, such as containers, are adopted on MEC hosts [20], as they offer increased portability, faster deployment, and low migration times. Furthermore, the MEC standard let applications interact with base stations of a cellular network via specific APIs [18].

Recently, the possible interplay between FL and MEC has been the subject of some investigations. The synergy of EC and FL algorithms promises to reduce the load on the network and to ensure privacy preservation during the collaborative training of AI models [21]. A taxonomy for FL solutions in EC environments has been proposed as well [21]. From an architectural point of view, a MEC host may act as the aggregator of local models for a restricted geographic area (instead of a cloud server), or as an intermediate layer in a hierarchical FL setting, where a higher level of MEC hosts federation is orchestrated by the cloud server. In a review paper on FL on Mobile Edge networks [22], research works on this subject are divided in two categories: FL *at* Mobile Edge Networks, and FL *for* Mobile Edge Networks. The former focuses on the main challenges of collaborative training of ML models in the edge environment, namely communication costs, resource allocation, and privacy and security aspects. The latter instead focuses on the adoption of FL to manage and optimize the network itself: the most relevant applications in this regard include computation offloading and content caching, malware and anomaly detection, and base station association.

Finally, we must recall that in some real applications FL is exploited for trustworthy AI systems: the most renowned case is the federated training of Deep Neural Networks for the improvement of query suggestions on Google Keyboard in Android devices⁵. However, to the best of our knowledge, specific issues in the deployment of applications supporting FL of XAI models over a MEC-enabled architecture have not been investigated so far.

III. THE PROPOSED FED-XAI APPLICATION

In this section we first describe the main functional modules that compose a Fed-XAI application offering trustworthy AI-

⁴https://www.etsi.org/deliver/etsi_gs/mec/001_099/003/01.01.01_60/gsmec003v010101p.pdf, last visited May 2023

⁵<https://ai.googleblog.com/2017/04/federated-learning-collaborative.html>, last visited May 2023

based services at the edge. Then, we provide some details regarding the implementation of the different modules and discuss the workflows underpinning such services.

In the Fed-XAI application we envision two main types of users, namely the *Application Administrator* and the *End User*, who gets access to the application through a dedicated User Equipment (UE). The UE is connected to a wireless network, such as a B5G/6G cellular network or an IoT network. The application will be deployed on a server, which may be a MEC host as discussed in Section II-B. AI-based services, like e.g. inference tasks, may be offered to the UE owners upon request. Examples of AI-based services include quality of experience prediction for video streaming applications [17] and intelligent decision-making in smart healthcare [21].

The application supports two main functionalities, namely training and inference. The training process is carried out according to the FL paradigm, as discussed in Section II-A: the involved participants learn a local XAI model, and a central entity is in charge of aggregating such models. In the inference process the resulting global model is exploited by an end-user to perform inference over its own private data.

A. Main Functional Modules

The different modules of the application can be grouped up into three macro-modules, each aimed to support specific functionalities: the *UE-related macro-module*, the *Centralized macro-module*, and the *Fed-XAI dashboard*, as indicated in the schematic view reported in Fig. 2.

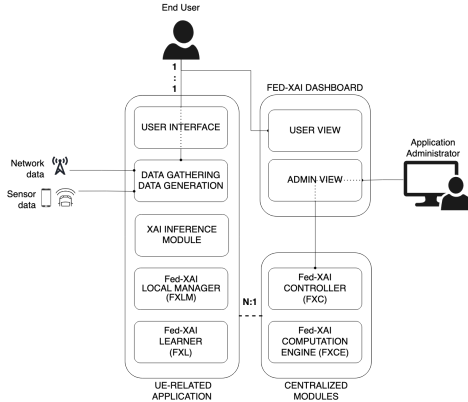


Fig. 2. Fed-XAI Application: Main Modules.

The UE-related macro-module is the portion of the system for carrying out the activities at the user place, so one instance is required per each user, and must be present in the corresponding UE. Its composing four modules allow a user to: i) interact with the application (User Interface module), ii) gather and generate data (Data Gathering - Data Generation module), iii) obtain predictions using an XAI model (XAI inference module), and iv) participate in the FL process (Fed-XAI local manager and Fed-XAI learner modules).

The Centralized macro-module manages the services to be offered to UEs, such as the UE registration, the prediction

task using an XAI model (Fed-XAI Controller), and the aggregation of XAI models (Fed-XAI Computation Engine).

The Fed-XAI dashboard provides both administration and end-user views. On one hand it allows the admin user to setup and control the FL environment; on the other hand, it provides the end-user with detailed information on the operation of XAI models, i.e., with the motivations of the obtained predictions, as we may expect from an interpretable XAI model. This information, in real-time or in the form of logs, can be made available to the administrator.

In the following we describe only the modules strictly related to the two main functionalities offered by the application, namely training (i.e., FL process) and inference:

- Fed-XAI Controller (FXC): the centralized control entity of the FL process; it is the core component of the application and manages the initialization and configuration of the federation, and offers a collection of XAI models to be exploited by UEs requesting a Fed-XAI service for the inference process.
- Fed-XAI Local Manager (FXLM): acts as a UE-related control entity of the FL process;
- Fed-XAI Learner (FXL): performs the training stage of the local models during the FL process, based on private end-user data.
- Fed-XAI Computation Engine (FXCE): aggregates the local models producing a global federated model, to be shared with the Fed-XAI Controller for persistent storing.
- XAI Inference Module: allows the end-user to exploit an XAI model for conducting inference on private data.
- Fed-XAI Dashboard: allows authenticated users to uncover model operation during inference.

The User Interface and the Data Gathering and Generation modules depend on the specific services offered to the UEs and should be appropriately implemented and deployed as a mobile application on the UE.

B. Implementation details

The modules of the proposed application have been designed and implemented to be compliant with an edge computing environment and exploit a fully virtualized architecture. In fact, each module is deployed inside a container of the Docker ecosystem⁶, which brings valuable benefits: first it allows portability of each module regardless of the underlying hardware and software infrastructure; second, it allows for migration in mobility scenarios [23].

Interaction among the different modules is implemented through messages exchanged via RESTful APIs over Secure HTTP protocol (HTTPS) as a security mechanism [24].

The FL process exploits the Intel[®] OpenFL⁷ library [25]. Although designed for the aggregation of models such as neural networks, i.e. via FedAvg or its variants, OpenFL offers high level of flexibility and customization. Specifically, we extended the library to support FL of inherently interpretable

⁶<https://www.docker.com/products/container-runtime>, last visited May 2023

⁷<https://github.com/intel/openfl>, last visited May 2023

models, such as TSK-FRBS, by implementing the procedure described in II-A. Within OpenFL, information about the configuration and execution of the FL process are specified in a collection of configuration files named *workspace*, which is delivered by the Fed-XAI Controller to all involved participants before the start of the FL process. Communications within OpenFL exploits the Transport Layer Secure (TLS) protocol.

The Fed-XAI dashboard has been implemented as a web application. Its backend logic offers a set of RESTful APIs to receive explainability information from the XAI Inference module. The frontend logic provides the web user interface.

A Repository module is implemented to persistently store trained models. To ensure fault tolerance and an adequate level of availability, we resort to MongoDB NoSQL database and, specifically, to its replication mechanism. The only module that needs to interact with the Repository module is the FXC.

C. Application workflow description

In this section we describe the workflow of the application with regard to the training process, based on FL, and the inference process.

Figure 3 reports a diagram of the modules involved in the training process, along with the interactions among them. A

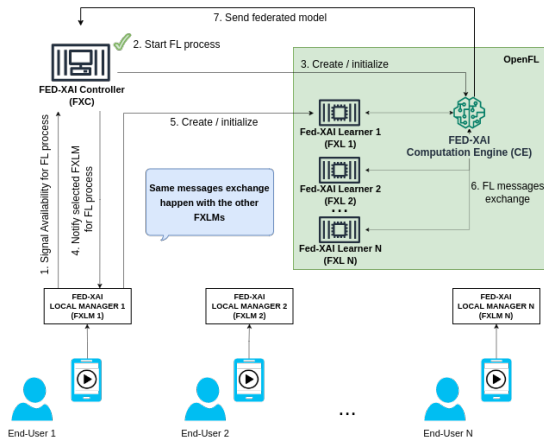


Fig. 3. Fed-XAI: Diagram of the training stage. Interactions among modules are represented as solid arrows

dedicated instance of FXLM is created for each end-user. The FXLM acts as the user proxy inside the application to provide the requested services to the end-user. Through the related FXLM, an end-user signals the availability to participate in the FL process to the FXC. Based on a specific policy, the FXC starts the FL process. In the implemented application we adopted a very simple policy: when the number of available UEs is higher than a fixed threshold, then the FXC initializes the FL process. Specifically, first it generates the OpenFL workspace. Then, it creates and initializes the FXCE by sharing the workspace along with the list of participants. Finally, it shares the workspace with the involved FXLM, possibly selecting only a subset of them. Then, each FXLM creates and initializes an FXL component in OpenFL, which takes part to the FL process as its UE-related entity. During

the initialization stage, the FXLM provides the FXL with the workspace and the private end-user data, to be used for training the local model. The actual FL process relies on the OpenFL library, suitably adapted to support XAI models: after the FXCE has verified the identity of the FXLMs and shared the configuration parameters, the FL procedure takes place in a single round. The final federated model, produced by the FXCE by aggregating the received local models, is sent to the FXC to be stored persistently in the Repository module. Once the FL process terminates, all the OpenFL containers terminate their execution.

Figure 4 shows the modules involved in the inference process, along with the sequence of the interactions among them. In this case, we suppose that a trained model is available in the Repository module. An end-user expresses

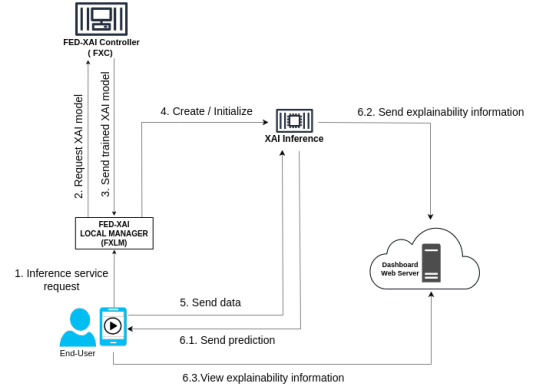


Fig. 4. Fed-XAI: Diagram of the inference stage. Interactions among modules are represented as solid arrows.

willingness to exploit the inference service to the relevant FXLM. The FXLM requests (and receives) a trained XAI model from the FXC, which fetches it from the Repository module. After receiving the XAI model, the FXLM creates and initializes (with the model itself) the XAI Inference module, which exposes a REST API endpoint for inference requests. Contextually, the end-user may connect to the Fed-XAI dashboard through browser in order to view the explainability information. Each novel input sample is transmitted from the UE to the XAI Inference module which replies with the associated prediction and feed the Fed-XAI dashboard with the explainability information. Whenever the end user quits from the inference service, the FXLM terminates the execution of the XAI Inference module.

IV. CASE STUDY

The proposed Fed-XAI application can be exploited in several areas, ranging from healthcare to smart industry: in this section, we discuss a case study in the automotive domain. First, a significant scenario is devised. Then, the application is tested under the reference scenario, illustrating the results.

A. Vehicular network case study

In the envisioned scenario, several vehicular UEs, connected to a B5G/6G network, receive and play a real-time video

stream. The perceived quality in video consumption is critical for the availability of advanced driving assistance technologies such as *see-through*, in which the driver of a vehicle can experience the live video acquired by the camera of a vehicle ahead to perform, e.g., a safer overtake maneuver. Indeed, it becomes crucial to be able to forecast, continuously over time, the video Quality of Experience (QoE) based on the contextual and Quality of Service (QoS) metrics available to the UE itself.

B. Fed-XAI Application Deployment for QoE Forecasting of Video Streaming

The first step is represented by the configuration and the instantiation of the container for the FXC. In this illustrative example, since we do not consider real UEs, the creation of FXLM containers is not prompted by requests from real end-users. Indeed, we manually instantiated 15 instances of FXLM, one for each UEs considered in the aforementioned simulation of a vehicular network scenarios. We assume that each FXLM knows the address of the FXC, and holds locally the simulated data collected by the vehicle of its end-user.

As per the training stage, we considered the FL of an XAI model, namely a TSK-FRBS, for video streaming QoE forecasting. A generic record of the training set for instant t and for a specific UE is obtained as follows: the input attributes are statistics calculated in a past time window of 10 seconds (i.e., over the interval $(t - 10, t]$) on a collection of QoS metrics extracted from the UE, whereas the target variable is the average value of QoE in the future horizon window of 1 second, (i.e., over the interval $(t, t + 1]$). We manually trigger the start of the FL process involving all the 15 UEs. The aggregated model was generated according to the procedure discussed in Section II-A, and stored in the Repository module. In this example, we obtained a TSK-FRBS model that achieved an average prediction capability, in terms of MSE and R^2 on a test set, equal to 0.066 and 0.559, respectively. Details on the experimental analysis and models comparison can be found in [28].

As per the inference stage, we simulated the request of QoE forecasting service by the UE to the FXC, through its FXLM. If a model for the inference service is available in the repository, it is returned to the FXLM, which is also in charge of instantiating a container for executing the specific inference module for QoE prediction task. Otherwise, the absence of a model for the requested service is notified to the FXLM. In a real deployment of our Fed-XAI application, we assume that an UE can feed the XAI inference container with data collected in real-time. In our simulated setting, instead, we consider a test set already available on the FXLM: the inference process is simulated through a sequence of requests in which the FXLM provides the inference container with the previously unseen data samples. After yielding each prediction, the XAI inference module sends explainability information, namely details about the rule responsible for the prediction, to the Fed-XAI Dashboard to let users understand the motivation behind the predicted value. Figure 5 reports a snapshot of the dashboard window during the inference stage.

Fig. 5. Snapshot of the Fed-XAI dashboard at inference stage.

- **QoE Prediction tab:** it shows the actual prediction (in the example, the predicted quality level of the video, expressed as a percentage) for an assigned horizon window, given a tuple of input statistics calculated on QoS metrics;
- **QoE Prediction History tab:** it tracks the most recent predicted QoE values;
- **Rule Antecedent tab:** it shows the linguistic value (“low”, “medium” or “high”, in the case of a fuzzy partitioning of an attribute domain with just three fuzzy sets) of each feature in the antecedent part of the activated rule. Due to the maximum matching strategy, the prediction in fact depends on a single rule (the one with the highest activation level);
- **Rule Consequent tab:** it shows the weights, namely the importance level, of each feature for current prediction, sorted by absolute value.

The obtained explainability information can be leveraged to identify potential actions to be taken. For example, the prediction can be exploited to turn on/off a driving assistance service. Moreover, the provided explanation may be relevant for different actors. On the end-user side, for instance, this outcome can be integrated into the navigation system as follows: if the degradation of the QoE depends on approaching an area with low coverage (thanks to inherent interpretability we can trace the cause of the decision), then an alternative route can be suggested where the coverage is sufficient.

V. CONCLUSION

In this paper we propose an application offering trustworthy AI-based services at the edge. Specifically, the application gives the possibility to an end-user to participate in the Federated Learning of Explainable AI models (Fed-XAI), and to exploit such models for performing inference on local private data. The application modules are designed to be deployed by means of lightweight virtualization techniques (i.e., containers), thus ensuring portability regardless of the underlying hardware and software infrastructure. Specifically, all modules can be deployed on Multi Access Edge Computing servers, which offer computing and storing capability closer to data sources, thus reducing latency and load on the network. We also illustrate a vehicular network case study in which the proposed Fed-XAI application is exploited in a simulated setting. A recently proposed federated approach to build first-order TSK fuzzy rule-based systems is implemented; at inference time, explainability information are displayed to the user through a dedicated dashboard.

ACKNOWLEDGMENT

Authors would like to thank the colleagues involved in Work Packages 4 and 5 of the Hexa-X project for the fruitful discussions on this activity.

REFERENCES

- [1] M. Filippou, V. Lamprousi, J. Mohammadi, M. Merluzzi, E. Ustundag Soykan, T. Borsos, N. Rajatheva, N. Rajapaksha, L. Le Magoarou, P. Piscione *et al.*, "Pervasive Artificial Intelligence in Next Generation Wireless: The Hexa-X Project Perspective," in *First Int'l WS on Artificial Intelligence in Beyond 5G and 6G Wireless Networks (AI6G 2022)*, 2022.
- [2] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," *Proc IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet Things*, vol. 3, no. 5, pp. 637–646, 2016.
- [4] F. Giust, X. Costa-Perez, and A. Reznik, "Multi-access edge computing: An overview of ETSI MEC ISG," *IEEE 5G Tech Focus*, vol. 1, no. 4, p. 4, 2017.
- [5] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, and T. Taleb, "Survey on Multi-Access Edge Computing for Internet of Things Realization," *IEEE Commun Surv Tut*, vol. 20, no. 4, pp. 2961–2991, 2018.
- [6] Y. Xiao, G. Shi, and M. Krunz, "Towards Ubiquitous AI in 6G with Federated Learning," 2020. [Online]. Available: <https://arxiv.org/abs/2004.13563>
- [7] A. Bandi, "A Review Towards AI Empowered 6G Communication Requirements, Applications, and Technologies in Mobile Edge Computing," in *2022 6th Int'l Conf on Computing Methodologies and Communication (ICCMC)*, 2022, pp. 12–17.
- [8] S. Lins, K. D. Pandl, H. Teigeler, S. Thiebes, C. Bayer, and A. Sunyaev, "Artificial Intelligence as a Service," *Bus & Inf Syst Eng*, vol. 63, no. 4, pp. 441–456, 2021.
- [9] E. Commission, C. Directorate-General for Communications Networks, and Technology, *Ethics guidelines for trustworthy AI*. Publications Office, 2019.
- [10] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans Intell Syst Technol (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.
- [11] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins *et al.*, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inform Fusion*, vol. 58, pp. 82–115, 2020.
- [12] J. L. Corcuera Bárcena, M. Daole, P. Ducange, F. Marcelloni, A. Renda, F. Ruffini, and A. Schiavo, "Fed-XAI: Federated Learning of Explainable Artificial Intelligence Models," in *XAI.it 2022: 3rd Italian WS on Explainable Artificial Intelligence, co-located with AI*IA 2022*, 2022.
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [14] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its applications to modeling and control," *IEEE T Sys Man Cyb*, no. 1, pp. 116–132, 1985.
- [15] A. Wilbik and P. Grefen, "Towards a Federated Fuzzy Learning System," in *2021 IEEE Int'l Conf on Fuzzy Systems (FUZZ-IEEE)*, 2021, pp. 1–6.
- [16] X. Zhu, D. Wang, W. Pedrycz, and Z. Li, "Horizontal federated learning of takagi–sugeno fuzzy rule-based models," *IEEE T Fuzzy Syst*, vol. 30, no. 9, pp. 3537–3547, 2022.
- [17] J. L. Corcuera Bárcena, P. Ducange, A. Ercolani, F. Marcelloni, and A. Renda, "An Approach to Federated Learning of Explainable Fuzzy Regression Models," in *2022 IEEE Int'l Conf. on Fuzzy Systems (FUZZ-IEEE)*, 2022, pp. 1–8.
- [18] E. Peltonen, M. Bennis, M. Capobianco, M. Debbah, A. Ding, F. Gil-Castiñeira, T. K. Marko Jurmu, M. Kelanti, A. Kliks, L. L. Teemu Leppänen, T. Mikkonen, A. Rao, S. Samarakoon, K. Seppänen, P. Sroka, and T. Y. Sasu Tarkoma, "6G white paper on Edge Intelligence," 2020. [Online]. Available: <https://arxiv.org/abs/2004.14850>
- [19] Y. Mansouri and M. A. Babar, "A review of edge computing: Features and resource virtualization," *J Parallel Distrib Comput*, vol. 150, pp. 155–183, 2021.
- [20] S. Chen and M. Zhou, "Evolving Container to Unikernel for Edge Computing and Applications in Process Industry," *Processes*, vol. 9, no. 2, 2021.
- [21] H. G. Abreha, M. Hayajneh, and M. A. Serhani, "Federated learning in edge computing: A systematic survey," *Sensors*, vol. 22, no. 2, 2022.
- [22] W. Y. B. Lim, N. C. Luong, D. T. Hoang, Y. Jiao, Y.-C. Liang, Q. Yang, D. Niyato, and C. Miao, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Commun Surv Tut*, vol. 22, no. 3, pp. 2031–2063, 2020.
- [23] L. Ma, S. Yi, and Q. Li, "Efficient Service Handoff across Edge Servers via Docker Container Migration," in *Proc of the 2nd ACM/IEEE Symp on Edge Computing*, ser. SEC '17. New York, NY, USA: ACM, 2017.
- [24] H. Subramanian and P. Raj, *Hands-On RESTful API Design Patterns and Best Practices: Design, develop, and deploy highly adaptable, scalable, and secure RESTful web APIs*. Packt Publishing Ltd, 2019.
- [25] P. Foley, M. J. Sheller, B. Edwards, S. Pati, W. Riviera, M. Sharma, P. N. Moorthy, S.-h. Wang, J. Martin, P. Mirhaji *et al.*, "OpenFL: the open federated learning library," *Phys Med Biol*, vol. 67, no. 21, p. 214001, 2022.
- [26] G. Nardini, D. Sabella, G. Stea, P. Thakkar, and A. Virdis, "Simu5G—An OMNeT++ Library for End-to-End Performance Evaluation of 5G Networks," *IEEE Access*, vol. 8, pp. 181 176–181 191, 2020.
- [27] J. L. Corcuera Bárcena, P. Ducange, F. Marcelloni, G. Nardini, A. Noferi, A. Renda, G. Stea, and A. Virdis, "Towards Trustworthy AI for QoE prediction in B5G/6G Networks," in *First Int'l WS on Artificial Intelligence in Beyond 5G and 6G Wireless Networks (AI6G 2022)*, 2022.
- [28] J. L. Corcuera Bárcena, P. Ducange, F. Marcelloni, G. Nardini, A. Noferi, A. Renda, F. Ruffini, A. Schiavo, G. Stea, and A. Virdis, "Enabling Federated Learning of Explainable AI Models within Beyond-5G/6G Networks," (under review).